

# CAHIER DES CHARGES

## ChatOps AI Enterprise – Automatisation DevOps de bout en bout pour un chatbot IA évolutif

### 1. Présentation du projet

#### 1.1 Contexte

Les entreprises recherchent aujourd'hui des solutions capables d'automatiser les interactions avec les utilisateurs tout en garantissant un haut niveau de disponibilité, de sécurité et d'observabilité.

Le projet ChatOps AI Enterprise vise à concevoir, déployer et exploiter une plateforme cloud-native permettant à des utilisateurs d'interagir avec une Intelligence Artificielle via une interface web moderne tout en appliquant les meilleures pratiques DevOps, DevSecOps et SRE.

#### 1.2 Objectifs

- Développer un chatbot basé sur l'IA générative.
  - Mettre en place une architecture cloud scalable.
  - Automatiser le déploiement via CI/CD.
  - Assurer la supervision et l'observabilité de la plateforme.
  - Garantir la résilience et la disponibilité du service.
  - Déployer l'ensemble sur une infrastructure AWS.
- 

## 2. Périmètre du projet

Le projet couvre :

### Frontend

- Développement d'une interface utilisateur moderne.
- Affichage des conversations.
- Tableau de bord de supervision.
- Communication avec l'API backend.

### Backend

- Exposition d'API REST FastAPI.
- Intégration avec l'API OpenAI.
- Gestion des requêtes utilisateur.
- Collecte de métriques Prometheus.

### Infrastructure

- Déploiement sur AWS EC2.
- Infrastructure as Code avec Terraform.

- Conteneurisation Docker.
- Orchestration Kubernetes.

### **Observabilité**

- Monitoring Prometheus.
- Dashboards Grafana.
- Centralisation des logs.
- Alertes e-mail via Alertmanager.

### **Automatisation**

- GitHub Actions CI/CD.
  - Build automatisé.
  - Déploiement automatisé.
- 

## **3. Architecture cible**

Utilisateur ↓ Frontend Next.js ↓ API FastAPI ↓ OpenAI API

Monitoring : Prometheus Grafana Alertmanager

CI/CD : GitHub Actions

Infrastructure : AWS EC2

Infrastructure as Code : Terraform

Containerisation : Docker

Orchestration : Kubernetes

---

## **4. Exigences fonctionnelles**

### **FR-01 Authentification utilisateur**

Le système doit permettre l'accès à l'application via une interface web sécurisée.

### **FR-02 Conversation IA**

L'utilisateur doit pouvoir :

- Saisir une question.
- Envoyer une requête.
- Recevoir une réponse générée par l'IA.

## FR-03 Historique des messages

Le système doit afficher les messages utilisateur et les réponses IA.

## FR-04 Tableau de bord

Le système doit afficher :

- CPU
- Mémoire
- État des services
- Nombre de requêtes

## FR-05 Supervision

Le système doit collecter et afficher les métriques techniques.

## FR-06 Alertes

Le système doit envoyer automatiquement des notifications e-mail en cas d'incident.

---

# 5. Exigences non fonctionnelles

## Performance

- Temps de réponse moyen inférieur à 2 secondes.
- Disponibilité supérieure à 99 %.

## Sécurité

- Gestion sécurisée des secrets.
- Protection des API.
- Limitation des accès réseau.

## Scalabilité

- Possibilité d'augmenter le nombre d'instances.
- Déploiement compatible Kubernetes.

## Maintenabilité

- Code versionné sous Git.
  - Documentation complète.
  - Déploiement automatisé.
-

## 6. Infrastructure technique

### Frontend

Technologie :

- Next.js
- TypeScript
- Tailwind CSS

Fonctions :

- Interface utilisateur
- Dashboard
- Chatbot

### Backend

Technologie :

- Python
- FastAPI

Fonctions :

- API REST
- Intégration OpenAI
- Export métriques Prometheus

### Base Cloud

Fournisseur :

- AWS

Services :

- EC2
  - Security Groups
  - Réseau sécurisé
-

## 7. Containerisation

### Docker

Conteneurs :

- Frontend
- Backend
- Prometheus
- Grafana
- Alertmanager

Objectifs :

- Portabilité
  - Reproductibilité
  - Isolation
- 

## 8. Kubernetes

Déploiement :

- Frontend Deployment
- Backend Deployment
- Services Kubernetes

Objectifs :

- Haute disponibilité
  - Scalabilité horizontale
  - Gestion des déploiements
- 

## 9. Infrastructure as Code

Outil :

Terraform

Fonctionnalités :

- Création automatique des ressources AWS
  - Gestion de l'infrastructure par code
  - Reproductibilité des environnements
-

## 10. CI/CD

Outil :

GitHub Actions

Pipeline CI :

- Vérification du code
- Tests automatisés
- Construction des images Docker

Pipeline CD :

- Déploiement automatique
  - Mise à jour des services
- 

## 11. Monitoring

### Prometheus

Collecte :

- État du backend
- Temps de réponse
- Disponibilité

### Grafana

Visualisation :

- CPU
- RAM
- Requêtes
- Disponibilité

### Alertmanager

Notifications :

- Backend indisponible
- Forte charge CPU
- Forte consommation mémoire
- Latence excessive

Canal :

- E-mail
- 

## 12. Plan de tests

### Tests unitaires

- Vérification des routes API.
- Vérification des réponses.

### Tests d'intégration

- Frontend ↔ Backend.
- Backend ↔ OpenAI.

### Tests d'exploitation

- Arrêt volontaire du backend.
  - Vérification du déclenchement des alertes.
  - Vérification de la réception des e-mails.
- 

## 13. Livrables

- Code source GitHub
  - Documentation technique
  - README professionnel
  - Fichiers Terraform
  - Manifests Kubernetes
  - Docker Compose
  - Dashboards Grafana
  - Configuration Prometheus
  - Configuration Alertmanager
  - Pipeline GitHub Actions
- 

## 14. Critères de réussite

Le projet sera considéré comme validé si :

- L'utilisateur peut dialoguer avec l'IA.
- Le déploiement est automatisé.
- Les services sont conteneurisés.
- L'infrastructure est provisionnée par Terraform.
- Les métriques sont visibles dans Grafana.

- Les alertes sont générées et reçues par e-mail.
  - Les pipelines CI/CD sont fonctionnels.
  - L'ensemble est déployé sur AWS.
- 

## 15. Conclusion

ChatOps AI Enterprise constitue une plateforme complète démontrant la maîtrise des compétences DevOps, Cloud Engineering, Infrastructure as Code, CI/CD, Observabilité, Conteneurisation et Intégration d'Intelligence Artificielle dans un environnement de production moderne.